AI Lab Practical Study Guide

Assignment: Supervised, Semi-Supervised and Ensemble Learning

1. Aim & Overview

To implement and compare supervised learning, semi-supervised learning, and ensemble learning methods for predicting a target outcome using a dataset. Our code compares three models:

- **Decision Tree** — Supervised learning
- **Self-Training Decision Tree** — Semi-supervised learning
- **Random Forest** — Ensemble learning

2. Libraries Used

```
import numpy as np from sklearn.datasets import make_classification from sklearn.model_selection import train_test_split from sklearn.tree import DecisionTreeClassifier from sklearn.ensemble import RandomForestClassifier from sklearn.semi_supervised import SelfTrainingClassifier from sklearn.metrics import accuracy_score
```

Library / Function	Purpose
`numpy`	Handles arrays and random operations
`make_classification`	Creates a sample classification dataset
`train_test_split`	Divides data into training and testing sets
`DecisionTreeClassifier`	Creates a decision tree model
`RandomForestClassifier`	Creates an ensemble of decision trees
`SelfTrainingClassifier`	Implements semi-supervised learning using pseudo-labels
`accuracy_score`	Measures prediction accuracy

3. Generate the Dataset

```
X, y = make_classification(n_samples=500, n_features=5, n_informative=4, n_redundant=0, random_state=42)
```

This creates a synthetic classification dataset with 500 records (`n_samples=500`), 5 input features (`n_features=5`), 4 informative features carrying useful prediction signals (`n_informative=4`), no redundant features (`n_redundant=0`), and `random_state=42` for reproducibility. **X** holds the input features; **y** contains target class labels (0 and 1).

4. Split the Dataset

```
      X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

Divides data into 80% training data (400 records to teach models) and 20% testing data (100 records to evaluate performance). `random_state=42` ensures a consistent split.

5. Model 1 — Supervised Learning (Decision Tree)

```
      model1 = DecisionTreeClassifier(random_state=42) model1.fit(X_train, y_train) pred1 =
model1.predict(X_test)
```

How it works: Creates a decision tree that learns threshold rules to split data into groups. `fit()` trains the model using known training features and labels (supervised learning). `predict()` classifies unseen test records.

6. Model 2 — Semi-Supervised Learning (Self-Training)

```
      y_semi = y_train.copy() rng = np.random.default_rng(42) y_semi[rng.random(len(y_semi))
< 0.5] = -1 model2 = SelfTrainingClassifier(DecisionTreeClassifier(random_state=42),
threshold=0.7) model2.fit(X_train, y_semi) pred2 = model2.predict(X_test)
```

How it works: Deliberately hides 50% of training labels by setting them to `-1` (unlabeled). Wraps a Decision Tree in a `SelfTrainingClassifier` with a `threshold=0.7`. It trains on labeled data, predicts labels for unlabeled data, accepts high-confidence predictions (≥70%) as *pseudo-labels*, and iteratively trains until convergence.

7. Model 3 — Ensemble Learning (Random Forest)

```
      model3 = RandomForestClassifier(n_estimators=100, random_state=42) model3.fit(X_train, y_train) pred3 = model3.predict(X_test)
```

How it works: Builds 100 decision trees (`n_estimators=100`) trained with data and feature randomness. Combines their predictions via majority voting (ensemble learning) to generalize better than a single tree.

8. Evaluation & 9. Complete Flow

```
      print("Decision Tree Accuracy:", accuracy_score(y_test, pred1)) print("Semi-supervised Accuracy:", accuracy_score(y_test, pred2)) print("Random Forest Accuracy:", accuracy_score(y_test, pred3))
```

Accuracy Formula: Accuracy = (Correct Predictions / Total Predictions) × 100. Evaluates fraction of correct test predictions (0 to 1 scale).

10. Key Viva Questions & Answers

Q1-Q3: *Supervised* uses known labels; *Semi-Supervised* uses mixed labeled/unlabeled data; *Ensemble* combines multiple models.
Q4-Q6: *Decision Tree* splits data like a flowchart; *Random Forest* combines 100 trees; *Pseudo-label* is a model-predicted training label.
Q7-Q12: *Split* separates training/testing; *fit()* trains; *predict()* infers; *accuracy* measures correctness; *random_state=42* ensures reproducibility; dataset is synthetic.